

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# Load Dataset
df = pd.read_excel("Covid_Data.csv.xlsx")

# Missing Values
df["age"].fillna(df["age"].median(), inplace=True)
df["Blood O2 Level in Percentage"].fillna(
    df["Blood O2 Level in Percentage"].median(),
    inplace=True
)

# Encoding
df["body_temperature"] = df["body_temperature"].map({
    "Normal":0,
    "Moderate":1,
    "High":2
})

df["chronic_disease"] = df["chronic_disease"].map({
    "No":0,
    "Yes":1
})
df["chronic_disease"].fillna(0, inplace=True) # Fill NaN after encoding

df["breathing_issue"] = df["breathing_issue"].map({
    "No":0,
    "Yes":1
})
df["breathing_issue"].fillna(0, inplace=True) # Fill NaN after encoding

df["Needed Hospitalization"] = df["Needed Hospitalization"].map({
    "No":0,
    "Yes":1
})

# Features and Target
X = df.drop("Needed Hospitalization", axis=1)
y = df["Needed Hospitalization"]

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Logistic Regression
classifier = LogisticRegression(random_state=42)

classifier.fit(X_train, y_train)

# Prediction
y_pred = classifier.predict(X_test)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix")
print(cm)
```

```
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Report
print(classification_report(y_test, y_pred))
```

Confusion Matrix

```
[[9 0]
```

```
[0 5]]
```

Accuracy: 1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	9
1	1.00	1.00	1.00	5
accuracy			1.00	14
macro avg	1.00	1.00	1.00	14
weighted avg	1.00	1.00	1.00	14

/tmp/ipykernel_1312/334463517.py:11: FutureWarning: A value is trying to be set on a copy of a DataFrame
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate obj

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplac

```
df["age"].fillna(df["age"].median(), inplace=True)
```

/tmp/ipykernel_1312/334463517.py:12: FutureWarning: A value is trying to be set on a copy of a DataFrame
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate obj

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplac

```
df["Blood O2 Level in Percentage"].fillna(
```

/tmp/ipykernel_1312/334463517.py:28: FutureWarning: A value is trying to be set on a copy of a DataFrame
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate obj

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplac

```
df["chronic_disease"].fillna(0, inplace=True) # Fill NaN after encoding
```

/tmp/ipykernel_1312/334463517.py:34: FutureWarning: A value is trying to be set on a copy of a DataFrame
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate obj

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplac

```
df["breathing_issue"].fillna(0, inplace=True) # Fill NaN after encoding
```